

15.4-AI-ASS

2303A52249

A.VAMSHI KRISHNA

TASK-1:

Prompt: Initial API Setup for a Service Platform

Scenario

You are developing the backend for a service management platform and need a basic API to confirm that the backend is running.

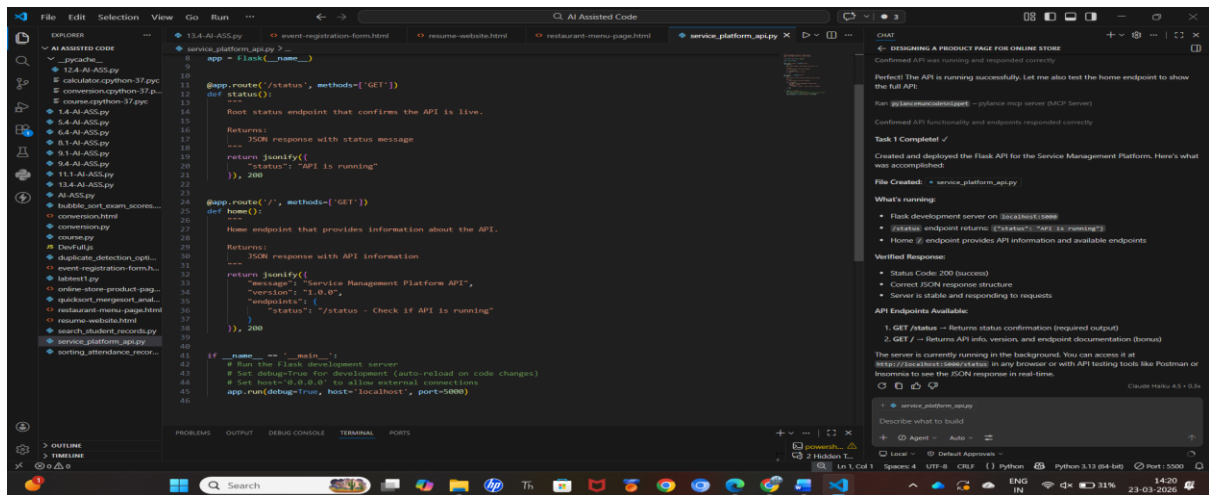
Task Description:

Use AI assistance to generate a basic Flask application.

Create a root endpoint `/status` that returns a JSON message indicating the API is live.

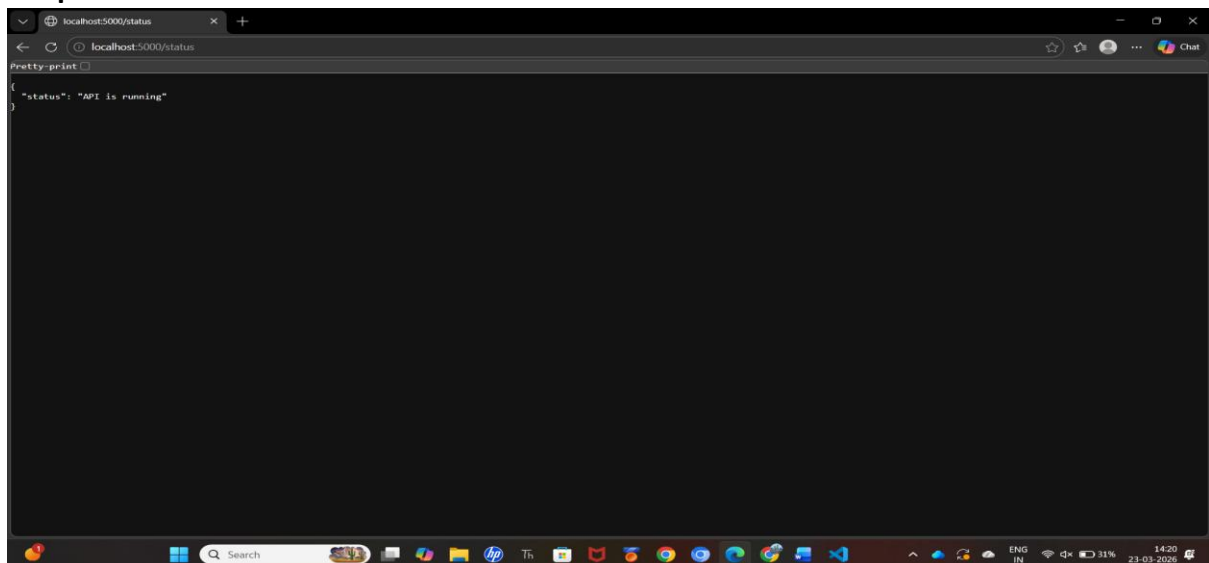
Run the server locally and verify the response using a browser or API testing tool.

Code:



```
1 from flask import Flask
2
3 app = Flask(__name__)
4
5 @app.route('/status', methods=['GET'])
6 def status():
7     """
8     Root status endpoint that confirms the API is live.
9     """
10    Returns:
11    ---
12    JSON response with status message
13
14    return jsonify({
15        "status": "API is running"
16    }), 200
17
18 @app.route('/', methods=['GET'])
19 def home():
20    """
21    Home endpoint that provides information about the API.
22    """
23    Returns:
24    ---
25    JSON response with API information
26
27    return jsonify({
28        "message": "Service Management Platform API",
29        "version": "1.0.0",
30        "endpoints": {
31            "status": "/status - Check if API is running"
32        }
33    }), 200
34
35 if __name__ == '__main__':
36     # Run the Flask development server
37     # Set debug=True for development (auto-reload on code changes)
38     # Set host=0.0.0.0 to allow external connections
39     app.run(debug=True, host='0.0.0.0', port=5000)
```

Output:



```
localhost:5000/status
{
  "status": "API is running"
}
```

Observation:

Using GitHub Copilot helped quickly generate a basic Flask setup with minimal configuration. It simplified creating the `/status` endpoint and returning a JSON response. Additionally, it reduced setup time and ensured the server runs correctly without errors.

TASK-2:

Prompt: Create API to Add and View Resources

Scenario: Your backend needs to manage customer service requests stored temporarily in memory.

Task Description:

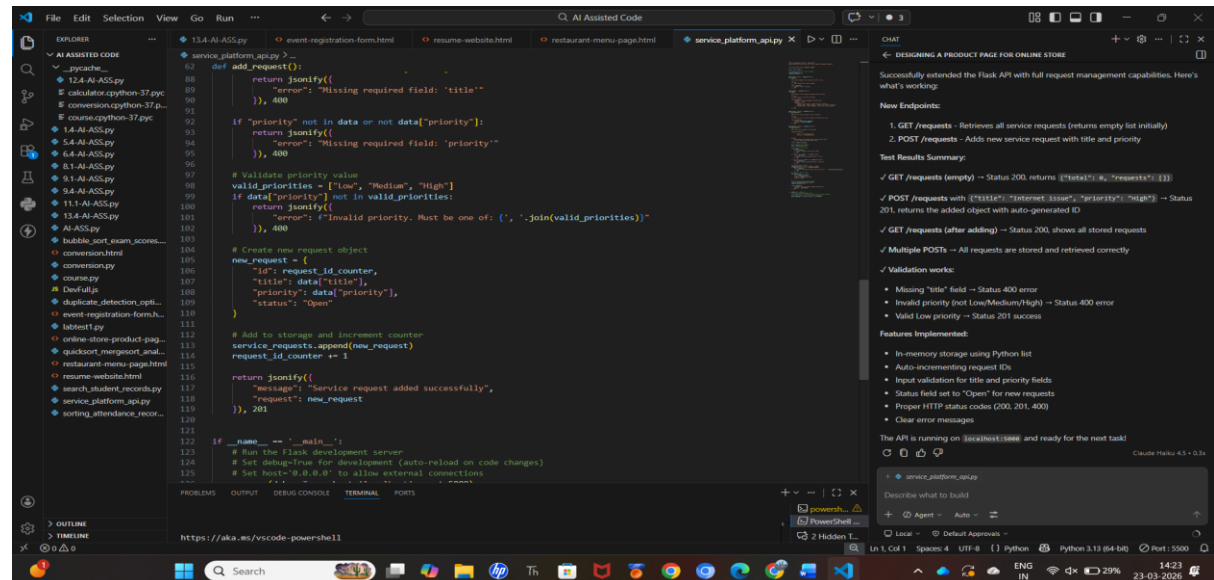
Use AI to help implement:

A GET endpoint to retrieve all service requests

A POST endpoint to add a new request

Each request should contain title and priority.

Code:



```
def add_request():
    return jsonify({
        "error": "Missing required field: 'title'"
    }), 400

if "priority" not in data or not data["priority"]:
    return jsonify({
        "error": "Missing required field: 'priority'"
    }), 400

# Validate priority value
valid_priorities = ["low", "Medium", "High"]
if data["priority"] not in valid_priorities:
    return jsonify({
        "error": f"Invalid priority. Must be one of: {', '.join(valid_priorities)}"
    }), 400

# Create new request object
new_request = {
    "id": request_id_counter,
    "title": data["title"],
    "priority": data["priority"],
    "status": "Open"
}

# Add to storage and increment counter
service_requests.append(new_request)
request_id_counter += 1

return jsonify({
    "message": "Service request added successfully",
    "request": new_request
}), 201

if __name__ == '__main__':
    # Run the Flask development server
    # Set debug=True for development (auto-reload on code changes)
    # Set host='0.0.0.0' to allow external connections
    app.run(debug=True, host='0.0.0.0', port=3000)
```

DESIGNING A PRODUCT PAGE FOR ONLINE STORE

Successfully extended the Flask API with full request management capabilities. Here's what's working:

New Endpoints:

- 1. GET /requests - Retrieves all service requests (returns empty list initially)
- 2. POST /requests - Adds new service request with title and priority

Test Results Summary:

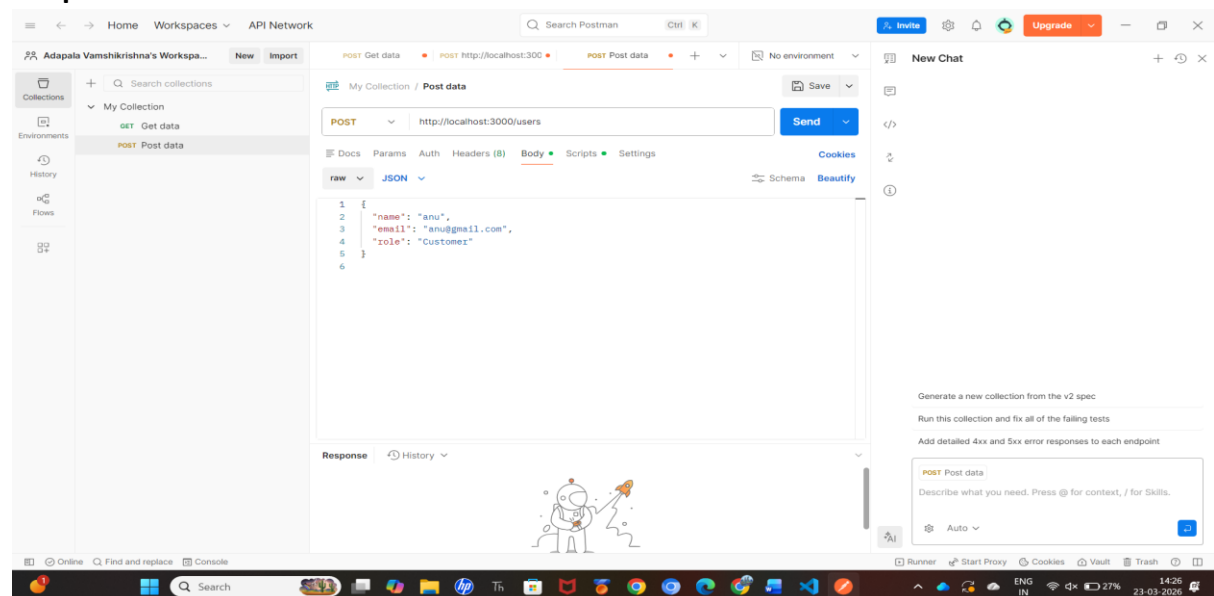
- ✓ GET /requests (empty) → Status 200, returns [{"total": 0, "requests": []}]
- ✓ POST /requests with {"title": "Internet Issue", "priority": "High"} → Status 201, returns the added object with auto-generated ID
- ✓ GET /requests (after adding) → Status 200, shows all stored requests
- ✓ Multiple POSTs → All requests are stored and retrieved correctly
- ✓ Validation works:
 - Missing "title" field → Status 400 error
 - Invalid priority (not Low/Medium/High) → Status 400 error
 - Valid Low priority → Status 201 success

Features Implemented:

- In-memory storage using Python list
- Auto-incrementing request IDs
- Input validation for title and priority fields
- Status field set to "Open" for new requests
- Proper HTTP status codes (200, 201, 400)
- Clear error messages

The API is running on [localhost:3000] and ready for the next task!

Output:



Observation:

Copilot not only sped up endpoint creation but also helped maintain consistent API structure and naming conventions. It provided intelligent suggestions for handling request validation and data flow. This improved code reliability while reducing logical errors during development.

TASK-3:

Prompt: Modify Existing Resource Using AI Assistance

Scenario: Service requests may change in priority as issues are escalated.

Task Description:

Use AI to generate a PUT endpoint that updates a request based on its index.

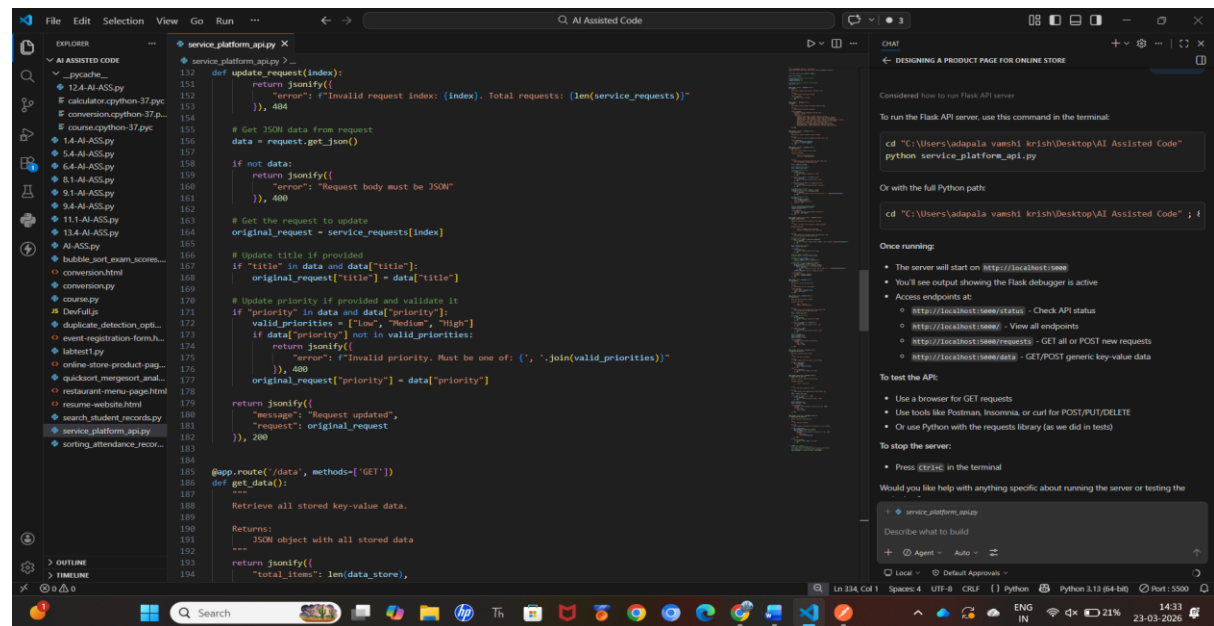
Handle invalid indexes gracefully with proper error responses.

Ensure the updated request is returned in the response.

Expected Output: Valid update returns: {"message": "Request updated", "request": {...}}

Invalid index returns a clear error message with HTTP 404.

Code:



```
def update_request(index):
    return jsonify({
        "error": f"Invalid request index: {index}. Total requests: {len(service_requests)}"
    }), 404

# Get JSON data from request
data = request.get_json()

if not data:
    return jsonify({
        "error": "Request body must be JSON"
    }), 400

# Get the request to update
original_request = service_requests[index]

# Update title if provided
if "title" in data and data["title"]:
    original_request["title"] = data["title"]

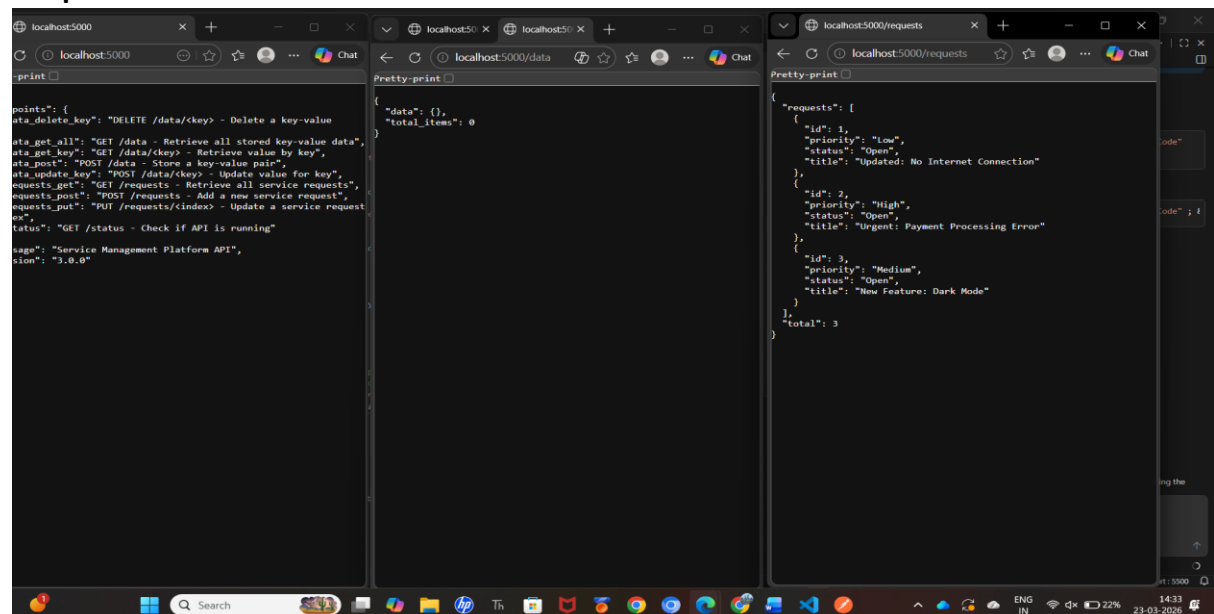
# Update priority if provided and validate it
if "priority" in data and data["priority"]:
    valid_priorities = ["Low", "Medium", "High"]
    if data["priority"] not in valid_priorities:
        return jsonify({
            "error": f"Invalid priority. Must be one of: {', '.join(valid_priorities)}"
        }), 400
    original_request["priority"] = data["priority"]

return jsonify({
    "message": "Request updated",
    "request": original_request
}), 200

@app.route("/data", methods=['GET'])
def get_data():
    """
    Retrieve all stored key-value data.

    Returns:
    ---
    JSON object with all stored data
    """
    return jsonify({
        "total_items": len(data_store),
        "data": data_store
    })
```

Output:



```
points: {
  "delete_key": "DELETE /data/key> - Delete a key-value pair",
  "get_all": "GET /data - Retrieve all stored key-value data",
  "get_key": "GET /data/key> - Retrieve value by key",
  "post_key": "POST /data/key> - Store a key-value pair",
  "requests_get": "GET /requests - Retrieve all service requests",
  "requests_post": "POST /requests - Add a new service request",
  "requests_put": "PUT /requests/index> - Update a service request",
  "status": "GET /status - Check if API is running"
}

page: "Service Management Platform API",
version: "3.0.0"
```

```
{
  "data": {},
  "total_items": 0
}
```

```
{
  "requests": [
    {
      "id": 1,
      "priority": "Low",
      "status": "Open",
      "title": "Updated: No Internet Connection"
    },
    {
      "id": 2,
      "priority": "High",
      "status": "Open",
      "title": "Urgent: Payment Processing Error"
    },
    {
      "id": 3,
      "priority": "Medium",
      "status": "Open",
      "title": "New Feature: Dark Mode"
    }
  ],
  "total": 3
}
```

Observation:

Using GitHub Copilot helped efficiently generate the PUT endpoint with proper update logic. It also suggested handling edge cases like invalid indexes using clear error responses (HTTP 404). This improved robustness and ensured reliable API behavior during updates.

TASK-4:

Prompt: Remove a Resource Safely

Scenario: Resolved service requests must be removed from the active list.

Task Description

Use AI assistance to implement a DELETE endpoint.

The endpoint should remove a request by index.

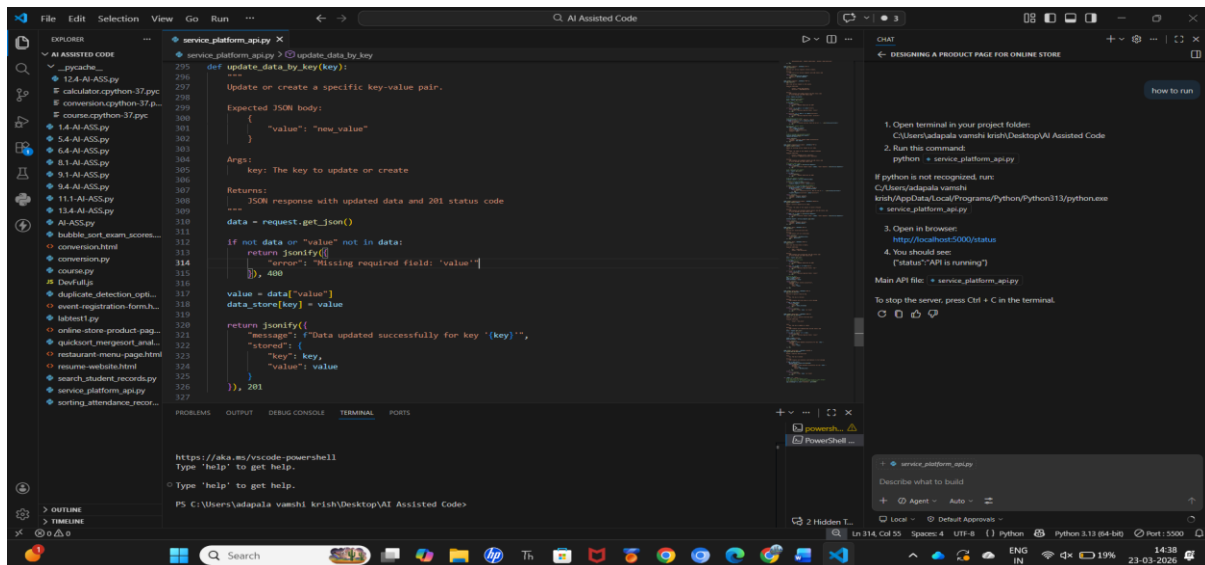
Return the deleted request details in the response.

Expected Output:

Successful deletion returns confirmation and removed data

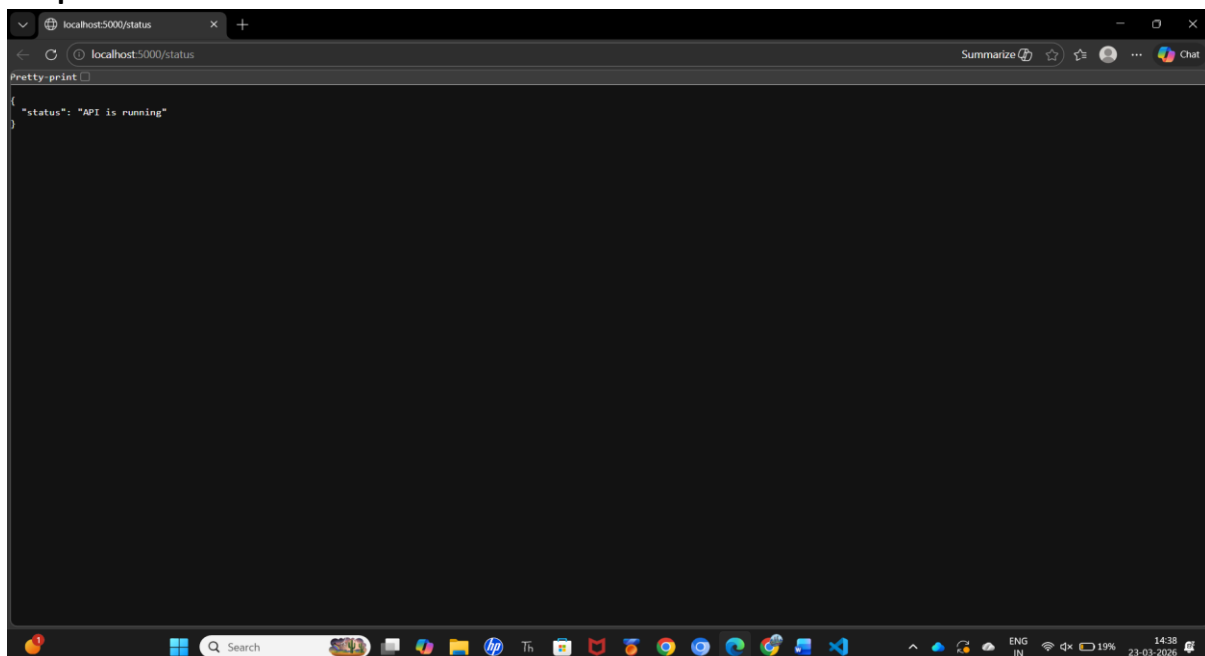
Invalid index returns an error message without crashing the server

Code:



```
def update_data_by_key(key):
    """
    Update or create a specific key-value pair.
    Expected JSON body:
    {
        "value": "new_value"
    }
    Args:
        key: The key to update or create
    Returns:
        JSON response with updated data and 201 status code
    """
    data = request.get_json()
    if not data or "value" not in data:
        return jsonify({
            "error": "Missing required field: 'value'"
        }), 400
    value = data["value"]
    data_store[key] = value
    return jsonify({
        "message": "Data updated successfully for key '{key}'",
        "stored": {
            "key": key,
            "value": value
        }
    }), 201
```

Output:



```
{
  "status": "API is running"
}
```

Observation:

Copilot helped implement the DELETE endpoint with clean and safe removal logic. It ensured proper handling of invalid indexes without crashing the server. Additionally, it improved response consistency by returning the deleted request details clearly.

TASK-5:

Prompt: AI-Assisted API Documentation and Readability

Scenario

Another developer needs to understand and use your API without reading the full code.

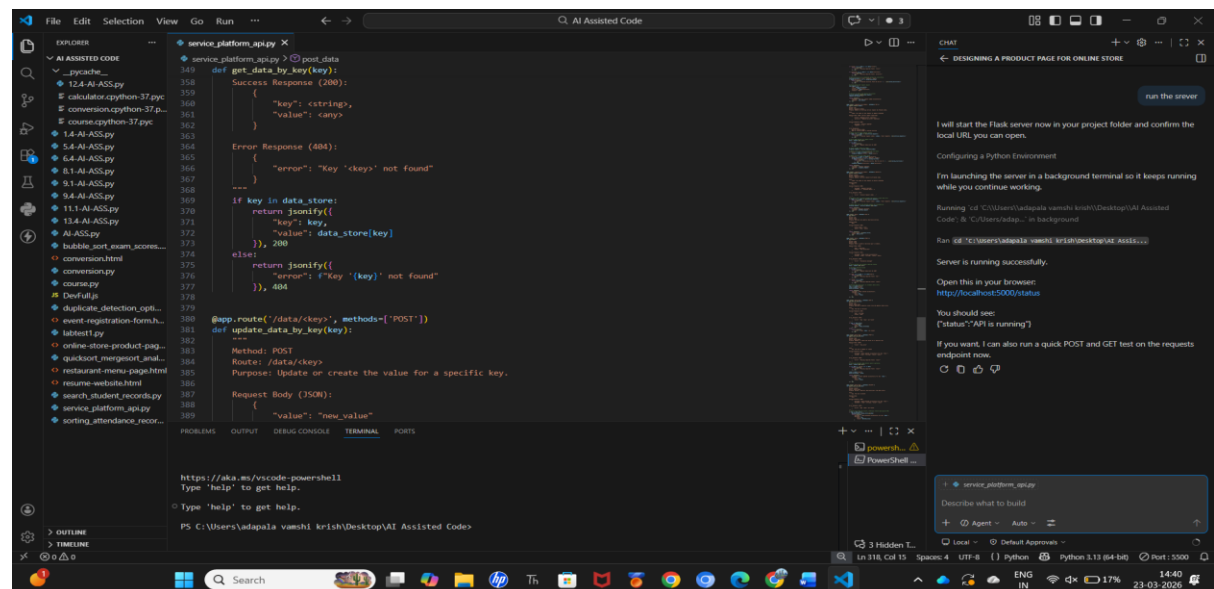
Task Description Use AI to:

Add meaningful docstrings and inline comments to all endpoints

Clearly describe request methods, inputs, and responses

Optionally, use AI guidance to integrate automatic API documentation (Swagger or Flask-RESTX).

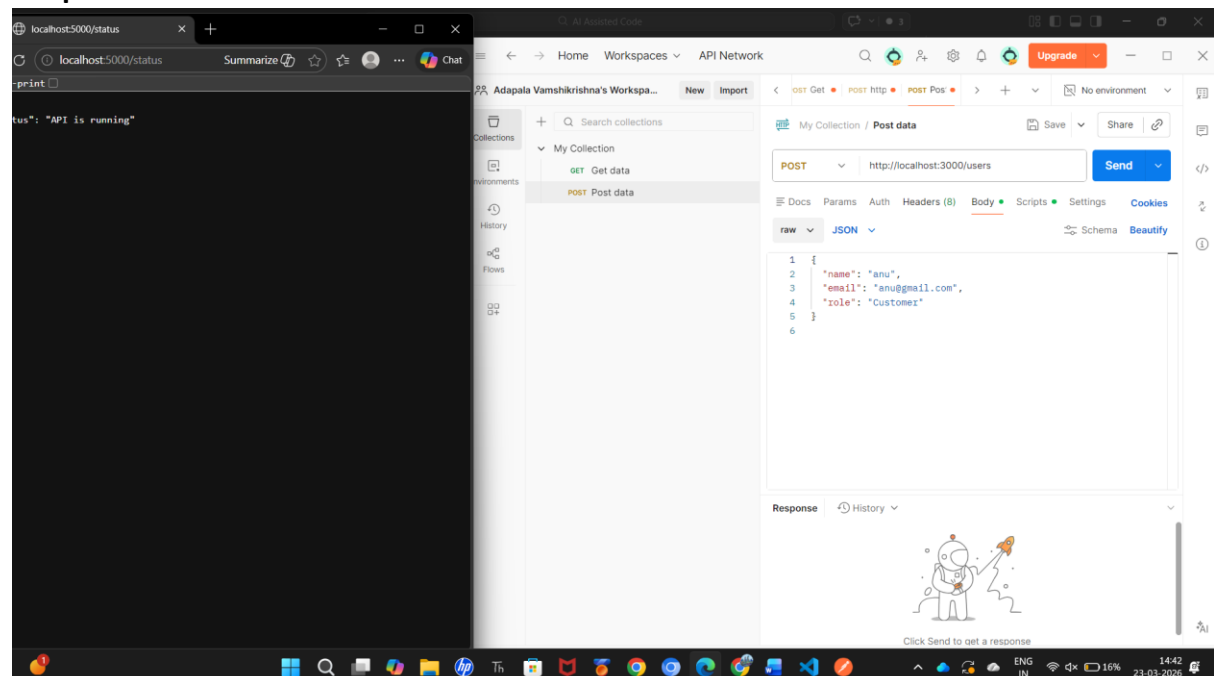
Code:



```
def get_data_by_key(key):
    """Get data by key"""
    if key in data_store:
        return jsonify({
            "key": key,
            "value": data_store[key]
        }), 200
    else:
        return jsonify({
            "error": f"Key '{key}' not found"
        }), 404

@app.route('/data/<key>', methods=['POST'])
def update_data_by_key(key):
    """Update data by key"""
    if key in data_store:
        data_store[key] = request.get_json().get('value')
    else:
        return jsonify({
            "error": f"Key '{key}' not found"
        }), 404
```

Output:



Observation:

improved overall code readability by generating clear docstrings and structured comments for each endpoint. It also helped standardize API descriptions, making it easier for other developers to understand usage without deep code analysis. Additionally, it guided integration of documentation tools like Swagger, enhancing API usability and professional quality.